

ASMETA

un ambiente per la specifica e l'analisi
di Abstract State Machines

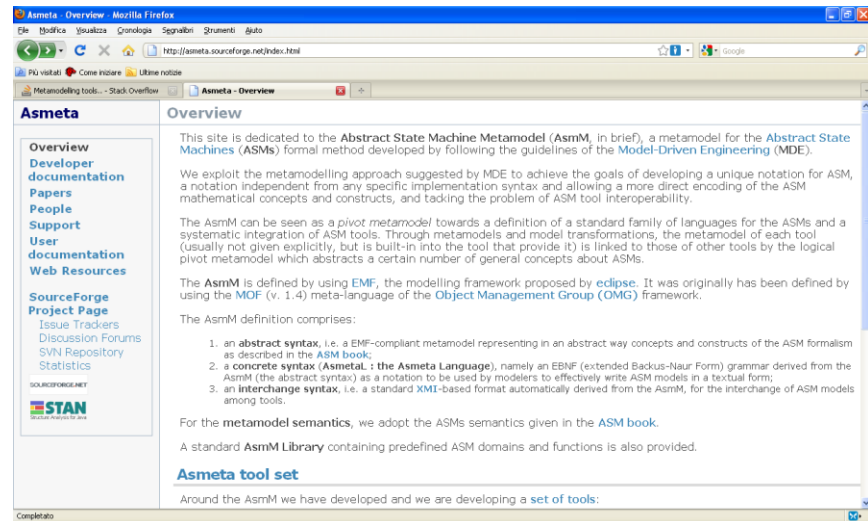


Elvinia Riccobene

elvinia.riccobene@unimi.it

ASMETA : ASm METAmodeling

- <https://asmeta.github.io/>
- Abstract State Machine Metamodel
 - tool set basato su Eclipse/EMF platform per MDE
- Model editing
 - e formato di interscambio
- Validation
 - simulazione
 - costruzione di *Scenari*
 - model-based testing
- Verification
 - static analisi
 - model checking



Gruppo di sviluppo di ASMETA



Angelo Gargantini

angelo.gargantini@unibg.it

Professore all'Università degli Studi di Bergamo



Elvinia Riccobene

elvinia.riccobene@unimi.it

Professore all'Università degli Studi di Milano



Patrizia Scandura

patrizia.scandurra@unibg.it

Professore Associato presso l'Università degli Studi di Bergamo



Paolo Arcaini

arcaini@nii.ac.jp

Professore Associato presso Istituto Nazionale di Informatica, Tokyo, Giappone



Silvia Bonfanti

silvia.bonfanti@unibg.it

RTDA presso l'Università degli Studi di Bergamo



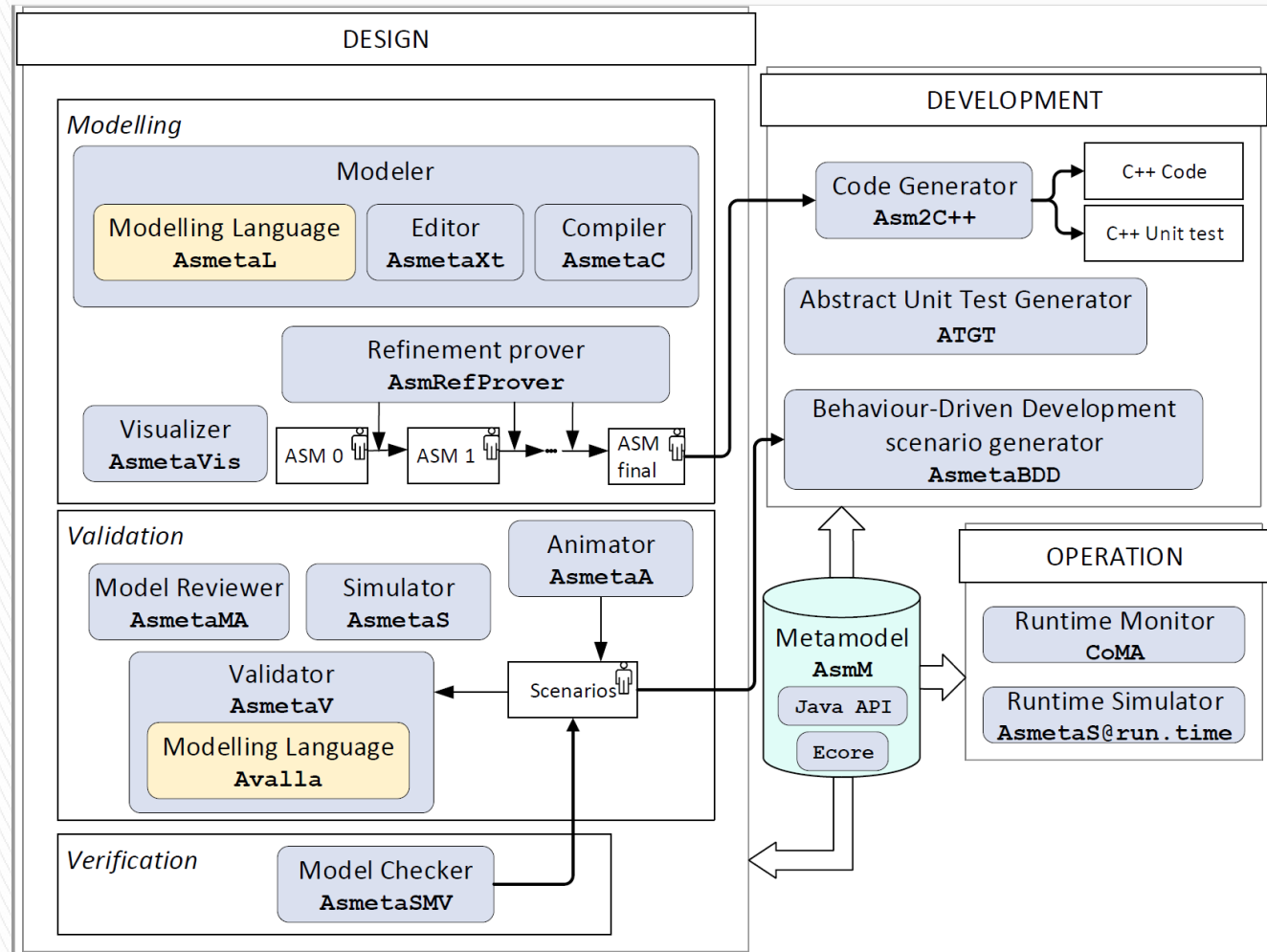
Andrea Bombarda

andrea.bombarda@unibg.it

RTDA presso l'Università degli Studi di Bergamo

+ molti studenti master

ASMETA-based Design Process

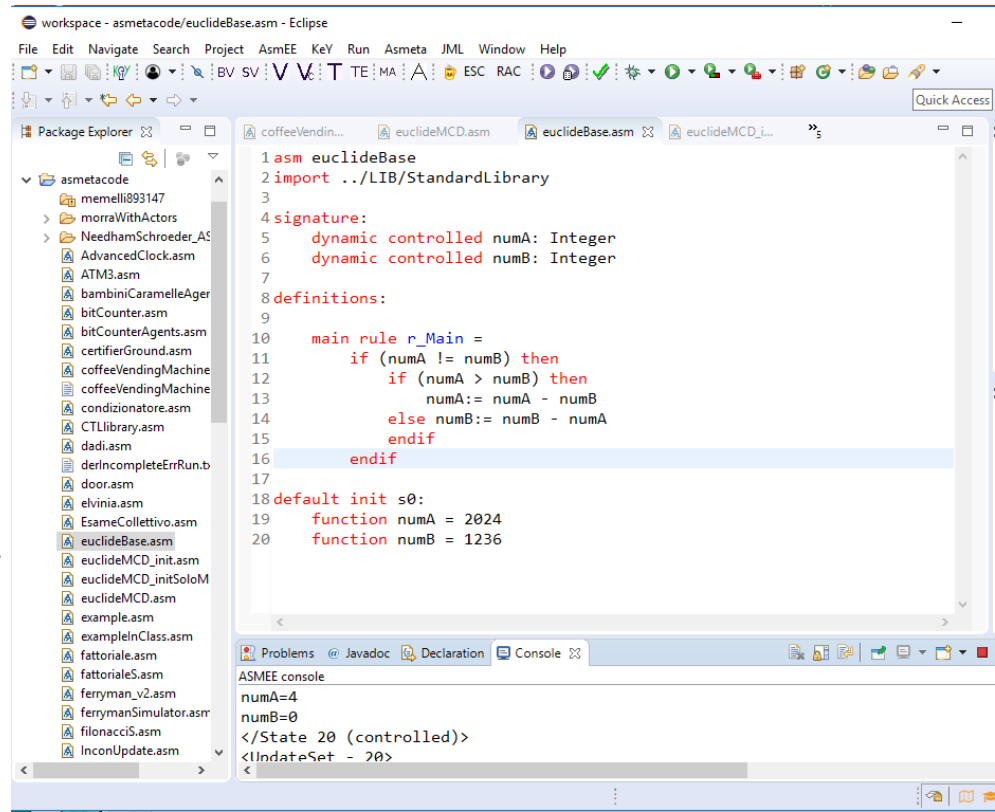


Installazione di ASMETA in Eclipse

- ❑ Eclipse plugin
- ❑ Vedere istruzioni in **Installing ASMETA in eclipse**
- ❑ Apri ASMETA....

Una versione standalone di Eclipse con tutti i plugin ASMETA è disponibile

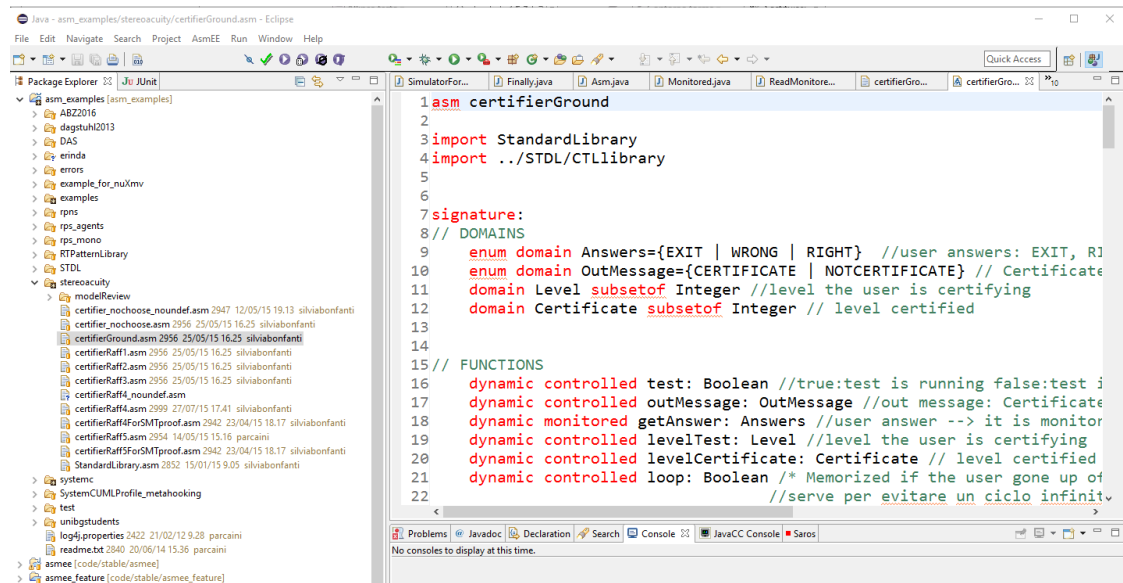
<https://github.com/asmeta/asmeta>



ASM Editor - ASMEE

ASMETA fornisce un editor integrato in eclipse

- simulazione, type checking possono essere eseguiti in ASMEE



```
1asm certifierGround
2
3import StandardLibrary
4import ../STDLib/CTLlibrary
5
6
7signature:
8// DOMAINS
9    enum domain Answers={EXIT | WRONG | RIGHT} //user answers: EXIT, RI
10    enum domain OutMessage={CERTIFICATE | NOTCERTIFICATE} // Certificate
11    domain Level subsetof Integer //level the user is certifying
12    domain Certificate subsetof Integer // level certified
13
14
15// FUNCTIONS
16    dynamic controlled test: Boolean //true:test is running false:test i
17    dynamic controlled outMessage: OutMessage //out message: Certificate
18    dynamic monitored getAnswer: Answers //user answer --> it is monitor
19    dynamic controlled levelTest: Level //level the user is certifying
20    dynamic controlled levelCertificate: Certificate // level certified
21    dynamic controlled loop: Boolean /* Memorized if the user gone up of
22                                     //serve per evitare un ciclo infinito
```

AsmetaL

il linguaggio testuale di ASMETA



Documentazione:

- [User document](#)
- [Quick guide](#)
- [EBNF grammar](#)
- **Tutorial** (pdf su myAriel)

Esempi al link:

https://github.com/asmeta/asmeta/tree/master/asm_examples

A. Gargantini, E. Riccobene, and P. Scandurra, "A Metamodel-based Language and a Simulation Engine for Abstract State Machines," J.UCS, vol. 14, no. 12, 2008.

Abstract State Machines

- Un modello ASM può essere letto come pseudocodice su strutture dati astratte
- ASM = (header, body, main, initialization)
 - **header**: signature declaration, import/export of modules/components
 - **body**: domains/functions defs., state invariant, rule decls.
 - **main rule**: punto di inizio del programma di una macchina
 - **initialization**: specifica un insieme di stati iniziali
 - uno stato iniziale definisce un valore iniziale per domini e funzioni dichiarate nella segnatura

Un primo esempio

Euclide MDC (internal values)

```
asm euclideMCD
import ../LIB/StandardLibrary
```

```
signature:
    dynamic controlled numA: Integer
    dynamic controlled numB: Integer
definitions:
```

```
main rule r_Main =
    if(numA != numB) then
        if(numA > numB) then
            numA := numA - numB
        else
            numB := numB - numA
        endif
    endif
```

```
default init s0:
function numA = 6409
function numB = 3289
```

} header

} body

} main rule

} initialization

Euclide MCD (monitored values)

```
asm euclideMCD_init
import ../LIB/StandardLibrary
signature:
    dynamic controlled numA: Integer
    dynamic controlled numB: Integer
    dynamic monitored numAinit: Integer
    dynamic monitored numBinit: Integer

definitions:
    main rule r_Main =
        if(numA != numB) then
            if(numA > numB) then numA := numA - numB
            else numB := numB - numA
            endif endif

default init s0:
    function numA = numAinit
    function numB = numBinit
```

Estensione di un file

- **asm** euclideanMCD
- uguale al file `.asm` (euclideanMCD.asm)
- **modulo** vs **asm**
 - **modulo** ASM è un ASM senza regola principale e inizializzazione

La libreria standard

- StandardLibrary.asm
- È un modulo contenente le definizioni di standard:
 - domini: Naturale, Intero, ...
 - funzioni: (+,- ...)
- Deve essere importato (nella header section):
`asm euclideMCD`
`import ../StandardLibrary`
`signature:`



Domini

type-domain:

- **basic type-domains**: definiti nella **standard library**!

`basic domain Complex`

`basic domain Real`

`basic domain Integer`

`basic domain Natural`

`basic domain Char`

`basic domain String`

`basic domain Boolean`

`basic domain Undef`

- **enum**: enumeration

`enum domain CoinType = {HALF | ONE}`

`enum domain Product = {COFFEE | TEA | MILK}`

Domini

- **abstract**: qualunque abstract domain

abstract domain NumCard

abstract domain Process

- definiti nella **standard library**:

abstract domain Agent

abstract domain Reserve

concrete domain: user-defined subset dei type-domain

domain QuantityDomain **subsetof** Integer

domain CoinDomain **subsetof** Integer

Advanced clock (esempio guida)

```
asm AdvancedClock
import ../LIB/StandardLibrary

signature:
domain Second subsetof Integer
domain Minute subsetof Integer
domain Hour subsetof Integer
controlled seconds: Second
controlled minutes: Minute
controlled hours: Hour

definitions:
domain Second = {0:59}
domain Minute = {0:59}
domain Hour = {0:23}

macro rule r_IncMinHours =
par
  if minutes = 59 then
    hours := (hours + 1) mod 24
  endif
  minutes := (minutes + 1) mod 60
endpar
```

```
main rule r_Main =
par
  if seconds = 59 then
    r_IncMinHours[]
  endif
  seconds := (seconds + 1) mod 60
endpar

default init s0:
function seconds = 0
function minutes = 0
function hours = 0
```

un clock che incrementa ad ogni passo i secondi e, se necessario, i minuti e le ore

Classificazione delle funzioni

- ▣ Static : funzioni invariate durante la run
- ▣ Dynamic : aggiornate durante la run
 - **Monitored** : modificato dall'ambiente (letto dalla macchina)
 - **Controlled** : modificato dalle regole come azioni dell'agente
 - **Shared** : modificato dall'ambiente e dalle regole
- ▣ Derived : definito da uno schema fisso in termini di altre funzioni (statiche o dinamiche).

Classificazione delle funzioni

```
asm euclideMCD_init
import ../LIB/StandardLibrary
signature:
    dynamic controlled numA: Integer
    dynamic controlled numB: Integer
    dynamic monitored numAinit: Integer
    dynamic monitored numBinit: Integer
```

```
asm coffeeVendingMachine
import ../../STDLib/StandardLibrary
signature:
    controlled available: Product -> QuantityDomain
    controlled coins: CoinDomain
    monitored insertedCoin: CoinType
    static num: Integer
```

Main rule

- `main rule R = rule`
- È il punto di partenza della computazione
 - Ricorda le altre regole
- R è il nome; rule è il body
- È una regola chiusa (no parametri)

```
main rule r_Main =  
    if(numA != numB) then  
        if(numA > numB) then numA := numA - numB  
        else numB := numB numA  
    endif endif
```

Inizializzazione

- È possibile definire **un insieme di stati iniziali**
 - Solo domini concreti dinamici
 - Solo funzioni controllate (no monitorate)
- Uno deve essere dichiarato come default

```
default init s0:  
  function numA = 6409  
  function numB = 3289
```

Convenzioni linguistiche

- **Variable ID** : inizia per "\$"
 - \$x \$abc \$door
- **ID_ENUM**: lettere maiuscole con lunghezza ≥ 2
 - ON OFF RED
- **ID_DOMAIN**: una stringa con lettera maiuscola
 - Integer Agent Persona
- **ID_RULE**: una stringa che inizia per "r_"
 - r_SetMyPerson r_update
- **ID_FUNCTION**: una stringa che inizia con una lettera minuscola (no: r_, inv_)
 - plus minus continue foo

Commenti

- Come in Java:
- `//` testo da commentare
- `/*` testo da commentare`*/`

Definizioni di dominio

□ Soltanto static concrete domains

domain MoneySize **subsetof** Integer

domain QuantityDomain **subsetof** Integer

domain CoinDomain **subsetof** Integer

definitions:

domain MoneySize = {10, 20, 40, 50, 100, 150, 200}

domain QuantityDomain = {0 : 10}

domain CoinDomain = {0 : 25}

coffeeVendingMachine.asm

Tic Tac Toe

```
asm ticTacToe_simulator
import ../LIB/StandardLibrary
signature:
    domain Coord subsetof Integer
    enum domain Sign = {CROSS | NOUGHT}
    enum domain Status = {TURN_USER | TURN_PC}

    controlled board: Prod(Coord, Coord) -> Sign
    controlled status: Status
    monitored userChoiceR: Coord
    monitored userChoiceC: Coord
    derived winner: Sign -> Boolean
    derived endOfGame: Boolean
```

Tic Tac Toe

definitions:

domain Coord = {1..3}

```
function winner($s in Sign) =  
  (exist $r in Coord with (forall $c in Coord with board($r,  
    $c) = $s)) or  
  (exist $c2 in Coord with (forall $r2 in Coord with board($r2,  
    $c2) = $s)) or  
  (forall $d in Coord with board($d, $d) = $s) or  
  (forall $d1 in Coord with board($d1, 4 - $d1) = $s)
```

```
function endOfGame =  
  (exist $s in Sign with winner($s)) or  
  (forall $r in Coord, $c in Coord with isDef(board($r, $c)))
```


Definizioni di funzioni

□ Solo static e derived **functions**

signature:

derived winner: Sign -> **Boolean**

derived endOfGame: **Boolean**

definitions:

```
function winner($s in Sign) =  
(exist $r in Coord with (forall $c in Coord with board($r, $c) = $s)) or  
(exist $c2 in Coord with (forall $r2 in Coord with board($r2, $c2) = $s)) or  
(forall $d in Coord with board($d, $d) = $s) or  
(forall $d1 in Coord with board($d1, 4 - $d1) = $s)
```

```
function endOfGame =  
(exist $s in Sign with winner($s)) or  
(forall $r in Coord, $c in Coord with isDef(board($r, $c)))
```

Definizioni di funzioni (da ATM)

□ Solo static e derived **functions**

signature:

```
abstract domain NumCard
static minMoney: Integer
static maxPrelievo: Integer
static pin: NumCard -> Integer
static card1: NumCard
static card2: NumCard
static card3: NumCard
```

definitions:

```
function minMoney = 200
function maxPrelievo = 1000
function pin($c in NumCard)
=
    switch($c)
        case card1 : 1
        case card2 : 2
        case card3 : 3
    endswitch
```

Costruttori di regole

Costruttori di regole semplici ma potenti :

□ Regola di transizione di base :

***if** Condition **then** Updates*

dove *Updates* è un insieme di aggiornamenti di funzioni
 $f(t_1, \dots, t_n) := t$ eseguiti simultaneamente quando
Condition è true

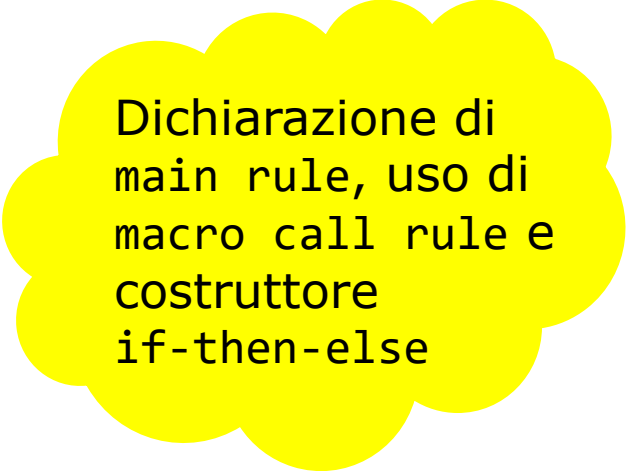
□ Esistono costruttori di regole più complessi :

- esecuzione condizionale (**if-the-else**)
- azioni parallele (**par**) e sequenziali (**seq**)
- estensione del dominio (**extend**)
- non determinismo (**choose**)
- unrestricted synch. parallelism (**forall**)
- (**let**)

Tic Tac Toe

```
main rule r_Main =  
  if(not(endOfGame)) then  
    if(status = TURN_USER) then  
      r_moveUser[]  
    else  
      r_movePC[]  
    endif  
  endif
```

```
default init s0:  
function status = TURN_USER
```



Dichiarazione di
main rule, uso di
macro call rule e
costruttore
if-then-else

Tic Tac Toe

```
rule r_moveUser =  
  let ($r = userChoiceR, $c = userChoiceC) in  
    if(isUndef(board($r, $c))) then  
      par  
        board($r, $c) := CROSS  
        status := TURN_PC  
      endpar  
    endif  
  endlet
```

Uso di costruttori:
if-then-else, let
par, choose

```
rule r_movePC =  
  choose $r in Coord, $c in Coord with isUndef(board($r, $c)) do  
    par  
      board($r, $c) := NOUGHT  
      status := TURN_USER  
    endpar
```

Esempio del ferryman (call rule)

```
asm ferrymanSimulator
```

```
import ../LIB/StandardLibrary
```

```
signature:
```

```
enum domain Actors = {FERRYMAN | GOAT | CABBAGE  
| WOLF}
```

```
enum domain Side = {LEFT | RIGHT}
```

```
dynamic controlled position: Actors -> Side
```

```
dynamic monitored carry: Actors
```

```
derived oppositeSide: Side -> Side
```

```
definitions:
```

```
function oppositeSide($s in Side) =  
  if($s = LEFT) then RIGHT  
  else LEFT  
endif
```

```
rule r_carry($a in Actors) =  
  if(position(FERRYMAN)=position($a)) then  
    par  
      position($a) := oppositeSide(position($a))  
      position(FERRYMAN) := oppositeSide(position(FERRYMAN))  
    endpar  
  endif
```

```
rule r_carry($a in Actors) =  
  if(position(FERRYMAN)=position($a)) then  
    par  
      position($a) := oppositeSide(position($a))  
      position(FERRYMAN) :=  
        oppositeSide(position(FERRYMAN))  
    endpar  
  endif
```

```
main rule r_Main = r_carry[carry]
```

```
default init s0:  
  function position($a in Actors) = LEFT
```

Fattoriale

dynamic monitored **valore**: Integer

dynamic controlled **indice**: Integer

dynamic controlled **fattoriale**: Integer

macro rule **fattor** =

 if (indice>1) then

 par

 fattoriale := fattoriale*indice

 indice := indice-1

 endpar

 endif



Uso del
seq

Fattoriale

main rule **main** =

seq

if (indice=1) then

if (valore>0) then

par

indice := valore

fattoriale := 1

endif

fattor[]

endseq

Inizializzazione:

default init s0:

function indice = 1



Fattoriale with stop

```
dynamic monitored valore: Integer
dynamic controlled indice: Integer
dynamic controlled fattoriale: Integer
dynamic controlled stop: Boolean
macro rule fattoriale =
  if (indice>1) then
    seq
      fattoriale := fattoriale*indice
      indice := indice-1
      if indice =1 then stop:=true
    endseq
  endif
```

Fattoriale with stop

```
main rule main =
```

```
if stop = false
```

```
then
```

```
  seq
```

```
    if (indice=1) then
```

```
      if (valore>0) then
```

```
        par
```

```
          indice := valore
```

```
          fattoriale := 1
```

```
        endif
```

```
      fattoriale[]
```

```
    endseq
```

Inizializzazione:

```
default init s0:
```

```
  function indice = 1
```

```
  function stop = false
```

Sorting di un vector

rule **swap**(x in Integer, y in Integer) =

par

x := y

y := x

endpar

asm SwapSort

asm swapBoard

rule **Sort** =

choose i in {0..9}, j in {0..9} with

i < j and vect(i) > vect(j) do

swap[vect(i), vect(j)]

Nota! non è necessaria alcuna variabile temporanea per scambiare due variabili

Per estendere i domini

- *Extend Rule:*

extend U **with** x [**do**] R

semantica : estende il dominio U con un nuovo valore x ed eseguire R

signature:

dynamic abstract domain Products

dynamic controlled inBuffer: Products

```
extend Products with $newProduct do
    r_writeBuffer[$newProduct]
```

```
rule r_writeBuffer($p in Products) =
    inBuffer := $p
```

parser



AsmataS

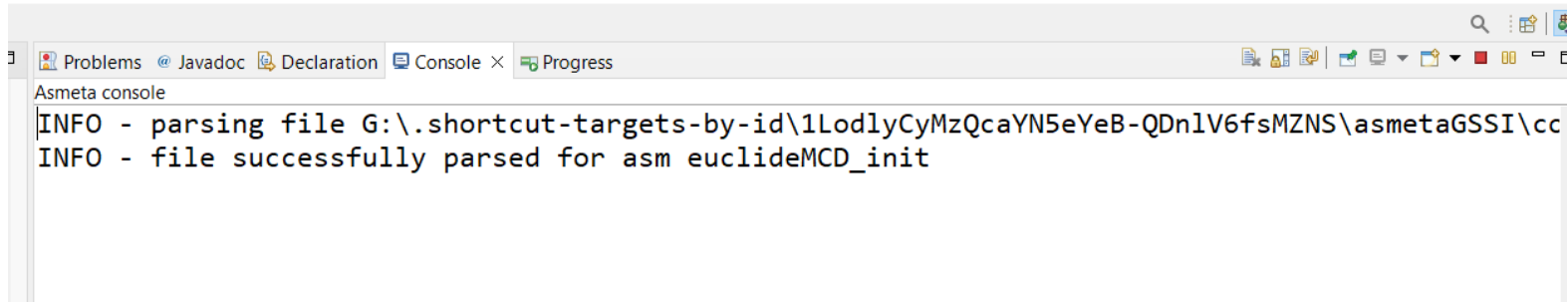
Analizzare un modello

 eclipse-workspace - asmetacode/euclideanMCD.asm - Eclipse IDE
File Edit Navigate Search Project AsmEE Run Asmeta Window Help



1. parser

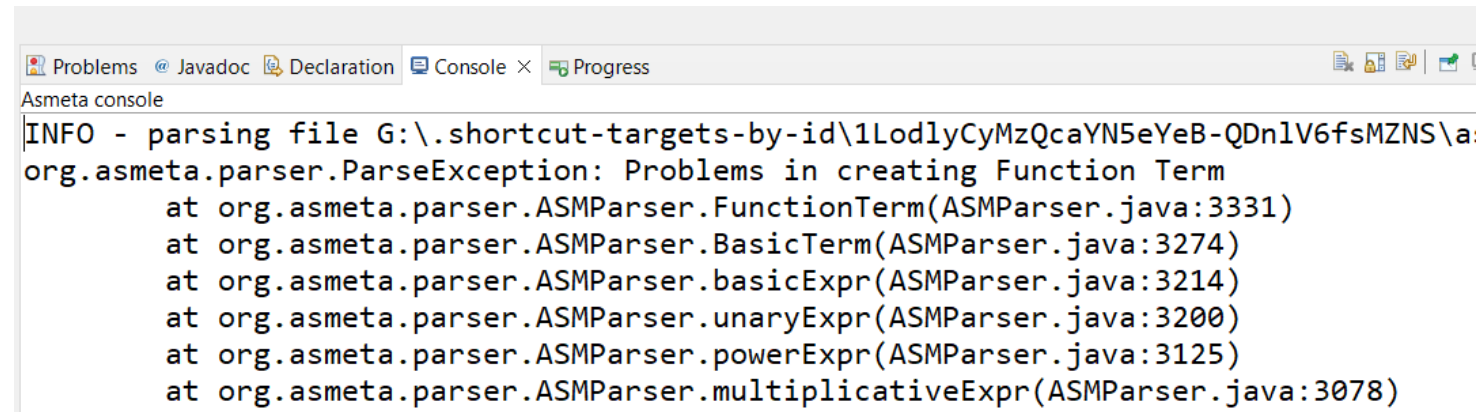
AsmetaL Console



The screenshot shows the AsmetaL IDE interface with the 'Console' tab selected. The console output displays two informational messages: 'INFO - parsing file G:\.shortcut-targets-by-id\1LodlyCyMzQcaYN5eYeB-QDn1V6fsMZNS\asmetaGSSI\cc' and 'INFO - file successfully parsed for asm euclideanMCD_init'.

```
Asmeta console
INFO - parsing file G:\.shortcut-targets-by-id\1LodlyCyMzQcaYN5eYeB-QDn1V6fsMZNS\asmetaGSSI\cc
INFO - file successfully parsed for asm euclideanMCD_init
```

□ Errors as violation of OCL constraints



The screenshot shows the AsmetaL IDE interface with the 'Console' tab selected. The console output displays an error message: 'org.asmeta.parser.ParseException: Problems in creating Function Term'. The stack trace includes the following entries:

```
Asmeta console
INFO - parsing file G:\.shortcut-targets-by-id\1LodlyCyMzQcaYN5eYeB-QDn1V6fsMZNS\asmetaGSSI\cc
org.asmeta.parser.ParseException: Problems in creating Function Term
    at org.asmeta.parser.ASMParser.FunctionTerm(ASMParser.java:3331)
    at org.asmeta.parser.ASMParser.BasicTerm(ASMParser.java:3274)
    at org.asmeta.parser.ASMParser.basicExpr(ASMParser.java:3214)
    at org.asmeta.parser.ASMParser.unaryExpr(ASMParser.java:3200)
    at org.asmeta.parser.ASMParser.powerExpr(ASMParser.java:3125)
    at org.asmeta.parser.ASMParser.multiplicativeExpr(ASMParser.java:3078)
```

Errore di parsing

Modello AsmetaL errato

```
asm example

import ./StandardLibrary

signature:
  controlled foo: Boolean
  monitored mon: Boolean

definitions:

  main rule r_main =
    foo := monA

default init s0:
  function foo = true
```

Errore del parser (monA non esiste)

```
INFO — parsing file example.asm
Problems locating the function monA
Exception: Function monA has not been found
org.asmeta.parser.ParseException: Problems in creating
                                   Function Term
    at org.asmeta.parser.ASMParser.FunctionTerm
                                   (ASMParser.java:3145)
    at org.asmeta.parser.ASMParser.BasicTerm
                                   (ASMParser.java:3087)
    at org.asmeta.parser.ASMParser.basicExpr
                                   (ASMParser.java:3030)
    at org.asmeta.parser.ASMParser.unaryExpr
                                   (ASMParser.java:3016)
    at org.asmeta.parser.ASMParser.powerExpr
                                   (ASMParser.java:2942)
    at org.asmeta.parser.ASMParser.multiplicativeExpr
                                   (ASMParser.java:2895)
    at org.asmeta.parser.ASMParser.additiveExpr
                                   (ASMParser.java:2853)
    at org.asmeta.parser.ASMParser.relationalExpr
                                   (ASMParser.java:2796)
```


simulator



AsmataS

Avviare il simulatore

 eclipse-workspace - asmetacode/euclideMCD.asm - Eclipse IDE
File Edit Navigate Search Project AsmEE Run Asmeta Window Help



Package Explorer



euclideM

- 2. Execution
 - Interactive
 - random

Simulation trace

INITIAL STATE:

<UpdateSet - 0>

numB=4

</UpdateSet>

<State 1 (controlled)>

numA=6

numB=4

</State 1 (controlled)>

<UpdateSet - 1>

numA=2

</UpdateSet>

<State 2 (controlled)>

numA=2

numB=4

</State 2 (controlled)>

<UpdateSet - 2>

numB=2

</UpdateSet>

<State 3 (controlled)>

numA=2

numB=2

</State 3 (controlled)>

<UpdateSet - 3>

</UpdateSet>

<State 4 (controlled)>

numA=2

numB=2

</State 4 (controlled)>

FINAL STATE:

numA=2

numB=2

run terminated

Funzionalità di AsmetaS

- **Consistent Updates checking**
per verificare la presenza di inconsistent updates
- **Random simulation**
valori casuali per le funzioni monitorate
 - *Euclide model*
- **Invariant checker**
Per controllare gli invarianti del modello

Simulazione interattiva

Modello AsmetaL

```
asm mons
import StandardLibrary

signature:
  enum domain EnumDom = {AA | BB
                        | CC}
  dynamic controlled foo: Integer
  dynamic monitored monB: Boolean
  dynamic monitored monE: EnumDom
  dynamic monitored monI: Integer

definitions:
  main rule r_main =
    if (monB) then
      if (monE = BB) then
        foo := monI
      endif
    endif
```

Simulazione

```
running mons.asm

Insert a boolean constant for monB:
true
Insert a symbol of EnumDom in [AA, BB, CC] for monE:
BB
Insert a integer constant for monI:
3
<State 0 (monitored)>
monB=true
monE=BB
monI=3</State 0 (monitored)>
<State 1 (controlled)>
foo=3</State 1 (controlled)>
Insert a boolean constant for monB:
true
Insert a symbol of EnumDom in [AA, BB, CC] for monE:
AA
<State 1 (monitored)>
monB=true
monE=AA</State 1 (monitored)>
<State 2 (controlled)>
foo=3</State 2 (controlled)>
```

Simulazione random

Modello AsmetaL

```
asm example

import ./StandardLibrary

signature:
  dynamic controlled foo: Boolean
  dynamic monitored mon: Boolean

definitions:

  main rule r_main =
    foo := mon

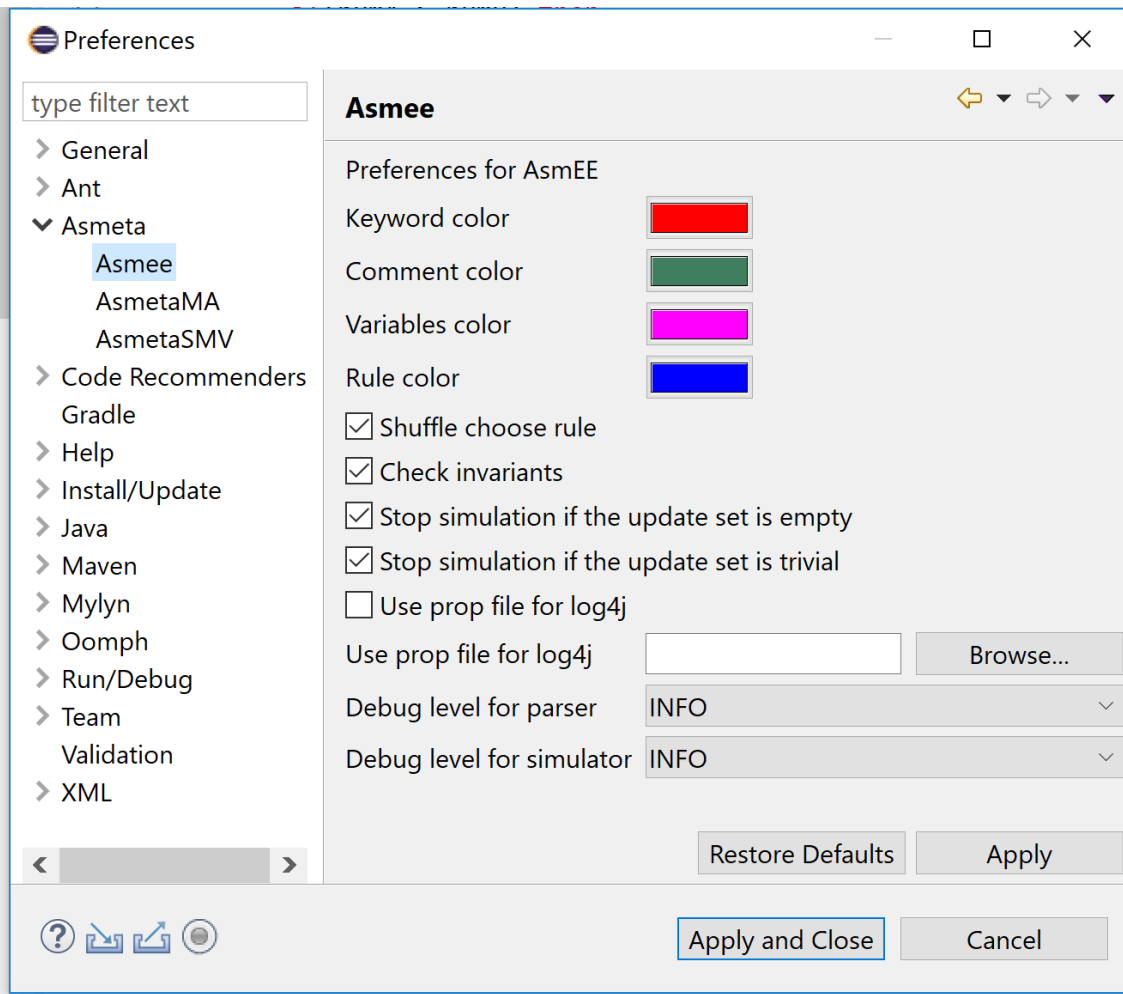
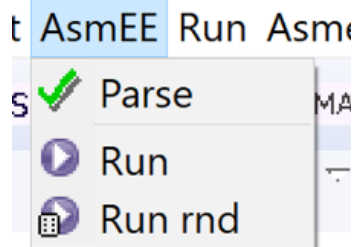
default init s0:
  function foo = true
```

Il simulatore sceglie in modo casuale i valori delle funzioni monitorate

Esecuzione con l'opzione **run rnd**

```
running rnd example.asm
taking false for mon
<State 0 (monitored)>
mon=false
</State 0 (monitored)>
<State 1 (controlled)>
foo=false
</State 1 (controlled)>
taking true for mon
<State 1 (monitored)>
mon=true
</State 1 (monitored)>
<State 2 (controlled)>
foo=true
</State 2 (controlled)>
Final state:
foo=true
```

Opzioni del Simulator



Opzione update set is empty

Modello AsmetaL

```
asm IncrementIf

import ./StandardLibrary

signature :
    dynamic controlled foo : Integer

definitions :

    main rule r_main =
        if (foo < 4) then
            foo := foo + 1
        endif

default init s0 :
    function foo = 0
```

Esecuzione con l'opzione **until empty**

```
running until empty IncrementIf.asm
<State 1 (controlled)>
foo=1
</State 1 (controlled)>
<State 2 (controlled)>
foo=2
</State 2 (controlled)>
<State 3 (controlled)>
foo=3
</State 3 (controlled)>
<State 4 (controlled)>
foo=4
</State 4 (controlled)>
<State 5 (controlled)>
foo=4
</State 5 (controlled)>
```


Opzione update set is empty

Modello AsmetaL

```
asm Invert

import ./StandardLibrary

signature:
    dynamic controlled foo: Boolean

definitions:

    main rule r_main =
        foo := not(foo)

default init s0:
    function foo = true
```

No stop!
Update set
mai empty

Esecuzione con l'opzione **until empty**

```
running until empty Invert.asm
<State 1 (controlled)>
foo=false
</State 1 (controlled)>
<State 2 (controlled)>
foo=true
</State 2 (controlled)>
<State 3 (controlled)>
foo=false
</State 3 (controlled)>
<State 4 (controlled)>
foo=true
</State 4 (controlled)>
<State 5 (controlled)>
foo=false
</State 5 (controlled)>
...
```

Opzione update set is trivial

```
1 asm trivialUpdate
2
3 import ../../STD/StandardLibrary
4
5 signature:
6   dynamic controlled foo: Integer
7
8 definitions:
9
10 main rule r_Main =
11   if foo < 4 then
12     foo := foo + 1
13   else
14     foo := 4
15   endif
16
17 default init s0:
18   function foo = 0
```

```
Asmeta console
simulating trivialUpdate.asm
INFO - parsing file C:\Users\elvinia\git\asmeta\asm_examples\
INFO - file successfully parsed for asm trivialUpdate
Running interactively trivialUpdate.asm
INITIAL STATE:
<UpdateSet - 0>
foo=1
</UpdateSet>
<State 1 (controlled)>
foo=1
</State 1 (controlled)>
<UpdateSet - 1>
foo=2
</UpdateSet>
<State 2 (controlled)>
foo=2
</State 2 (controlled)>
<UpdateSet - 2>
foo=3
</UpdateSet>
<State 3 (controlled)>
foo=3
</State 3 (controlled)>
<UpdateSet - 3>
foo=4
</UpdateSet>
<State 4 (controlled)>
foo=4
</State 4 (controlled)>
<UpdateSet - 4>
foo=4
</UpdateSet>
<State 5 (controlled)>
foo=4
</State 5 (controlled)>
FINAL STATE: foo=4
run terminated
```

Opzione shuffle

Modello AsmetaL

```
asm nonDet

import ./StandardLibrary

signature:
  enum domain EnumDom = {AA | BB | CC}
  dynamic controlled foo: EnumDom

definitions:

  main rule r_main =
    choose $x in EnumDom with $x != BB do
      foo := $x

  default init s0:
    function foo = BB
```

Esecuzione con l'opzione **shuffle**

```
running nonDet.asm
<State 1 (controlled)>
foo=AA
</State 1 (controlled)>
<State 2 (controlled)>
foo=CC
</State 2 (controlled)>
<State 3 (controlled)>
foo=AA
</State 3 (controlled)>
<State 4 (controlled)>
foo=AA
</State 4 (controlled)>
...
```

Il simulatore abilita la scelta
non deterministica nelle
choose rule

Aggiornamenti inconsistenti

Modello AsmetaL

```
asm InconUpdate

import ./StandardLibrary

signature:
    dynamic controlled foo: Integer
    dynamic monitored monA: Boolean
    dynamic monitored monB: Boolean

definitions:
    main rule r_main =
        par
            if (monA) then
                foo := 2
            endif
            if (monB) then
                foo := 3
            endif
        endpar
```

Errore in simulazione

```
running InconUpdate.asm

Insert a boolean constant for monA:
true
Insert a boolean constant for monB:
true
INCONSISTENT UPDATE FOUND !!! :
location foo updated to 2 != 3
Final state:
monA=true
monB=true
```

Esempio di aggiornamenti inconsistenti

abstract domain **Orders**

enum domain **Status** = {PENDING | INVOICED | CANCEL}

static o1: Orders

static o2: Orders

controlled orderStatus: Orders -> Status

main rule **main** =

par

choose \$o in Orders with true do

orderStatus(\$o) := INVOICED

choose \$oo in Orders with true do

orderStatus(\$oo) := CANCEL

endpar

default init s0:

function orderStatus(\$o in Orders) = PENDING

Aggiornamenti inconsistenti

Modello AsmetaL

```
asm OrderSystemInconUpd
import ./StandardLibrary

signature:
  abstract domain Orders
  enum domain Status = {PENDING |
                        INVOICED | CANCELLED}

  static o1: Orders
  static o2: Orders
  controlled status: Orders -> Status

definitions:
  main rule r_main =
    par
      choose $o in Orders with true do
        status($o) := INVOICED
      choose $oo in Orders with true do
        status($oo) := CANCELLED
    endpar

  default init s0:
    function status($o in Orders) = PENDING
```

Errore in simulazione

```
running (-shuffle) OrderSystemInconUpd.asm

<State 1 (controlled)>
Orders={o1,o2}
status(o1)=INVOICED
status(o2)=CANCELLED
</State 1 (controlled)>
<State 2 (controlled)>
Orders={o1,o2}
status(o1)=CANCELLED
status(o2)=INVOICED
</State 2 (controlled)>
<State 3 (controlled)>
Orders={o1,o2}
status(o1)=INVOICED
status(o2)=CANCELLED
</State 3 (controlled)>
INCONSISTENT UPDATE FOUND !!! : location
status(o2) updated to INVOICED != CANCELLED
FINAL STATE: Orders={o1,o2}
run terminated
```

Valore undef

Modello AsmetaL

```
asm undefValue
import ./StandardLibrary

signature :
    dynamic controlled fooA: Integer
    dynamic controlled fooB: Integer

definitions :
    main rule r_Main =
        par
            if (isUndef(fooA)) then
                fooA := 1
            endif
            if (isDef(fooB)) then
                fooB := fooB + 1
            endif
        endpar

    default init s0:
        function fooB = 2
```

Simulazione

```
running undefValue.asm
<State 1 (controlled)>fooA=1
fooB=3</State 1 (controlled)>
<State 2 (controlled)>fooA=1
fooB=4</State 2 (controlled)>
<State 3 (controlled)>fooA=1
fooB=5</State 3 (controlled)>
Final state:
fooA=1
fooB=5
```

Osservazioni

- ▶ La funzione booleana *isUndef* della StandardLibrary controlla se il valore di una locazione è *undef*.
- ▶ Nello stato iniziale *fooA* vale *undef*.

Valore undef (2)

Modello AsmetaL

```
asm undefValue2
import ./StandardLibrary

signature:
    dynamic controlled fooA: Integer
    dynamic controlled fooB: Integer

definitions:
    main rule r_Main =
        par
            if (isDef(fooA)) then
                fooA := undef
            endif
            if (isUndef(fooA)) then
                fooB := fooB + 1
            endif
        endpar

default init s0:
    function fooA = 1
    function fooB = 2
```

Simulazione

```
running undefValue2.asm
<State 1 (controlled)>fooA=undef
</State 1 (controlled)>
<State 2 (controlled)>fooA=undef
fooB=3</State 2 (controlled)>
<State 3 (controlled)>fooA=undef
fooB=4</State 3 (controlled)>
Final state:
fooA=undef
fooB=4
```

Osservazione

È possibile aggiornare in modo esplicito una locazione al valore *undef* (nell'esempio *fooA*).

Esempi

- Euclide MCD
- Ferryman
- Fattoriale
- Tic Tac Toe

animator



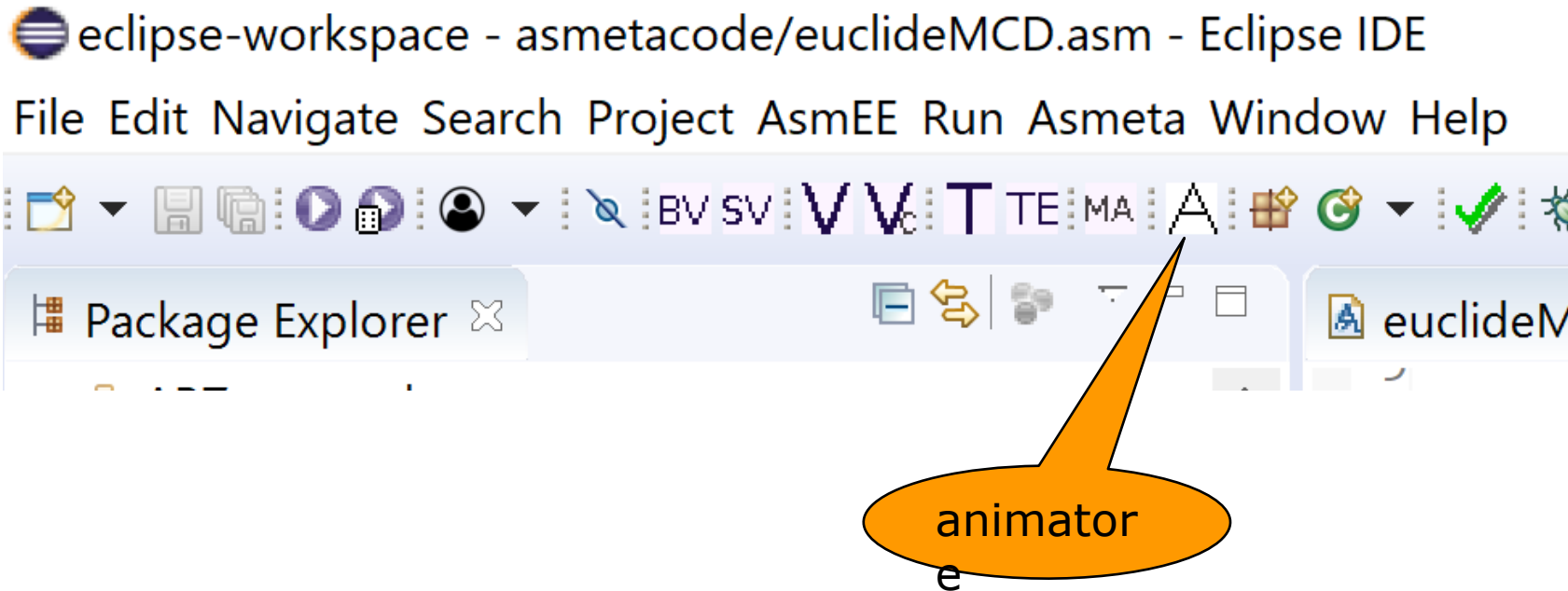
AsmataA

S. Bonfanti Silvia, A. Gargantini, A. Mashkoor:
AsmetaA: Animator for Abstract State Machines.
ABZ 2018, LNCS, 2018

ASMETA Animator

Rappresentazione grafica di una simulazione

Avvia l'animatore



ASMETA Animator

AsmetaA

Do one interactive step

Do random step/s

Insert random step number

1

Invariant violation / exceptions

Move Controlled Functions U

Move Controlled Functions DO

Move Monitored Functions U

Move Monitored Functions DO

export to Avalla

	Type	Functions	State 0	State 1

	Type	Functions	State 0	State 1
☐	M	numAinit	24	
☐	M	numBinit	18	
☐	C	numA	24	6
☐	C	numB	18	18

```
<State 0 (monitored)>
numAinit=24
numBinit=18
</State 0 (monitored)>
<UpdateSet - 0>
numA=6
</UpdateSet>
<State 1 (controlled)>
numA=6
numB=18
</State 1 (controlled)>
```

Uso dell'Animatore

- Animare un'esecuzione ASM
 - euclideMCD.asm
 - traghettatoreSimulator.asm

visualizer



AsmataVis

P. Arcaini, S. Bonfanti, A. Gargantini, E. Riccobene.

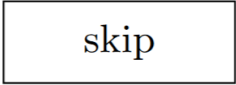
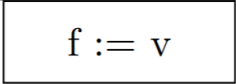
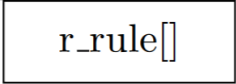
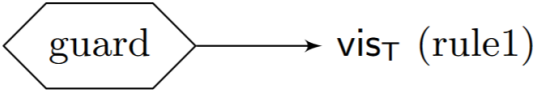
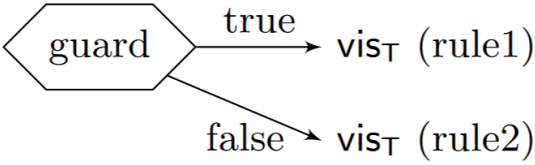
Visual notation and patterns for Abstract State Machines.

In Proc of Human-Oriented Formal Methods (HOFM 2016), 2016

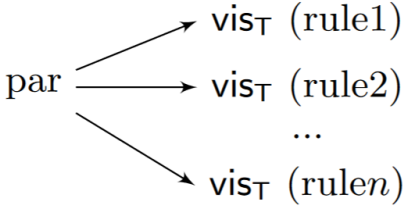
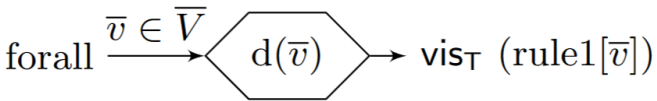
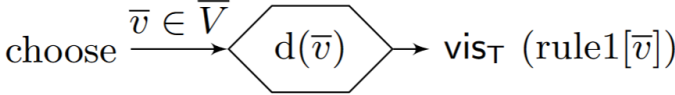
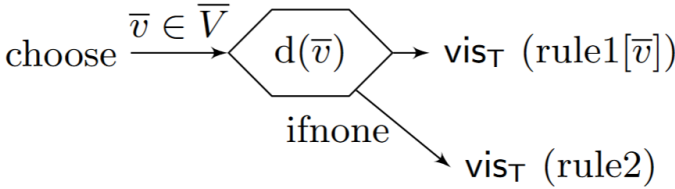
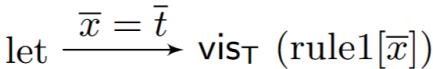
Visualizzazione del modello

- Visualizza un modello in termini di una foresta navigabile di **strutture ad albero**
- Si applica:
 - **Modelli strutturali**
 - utile per visualizzare la struttura del modello in modo compatto
 - **Modelli semantici**
 - Utile quando dal modello è possibile dedurre ulteriori informazioni sul flusso di lavoro della macchina
 - Permette una visualizzazione del modello in termini di macchina a stati finali quando una funzione *di modo* è definita e cambia di stato

Notazione grafica

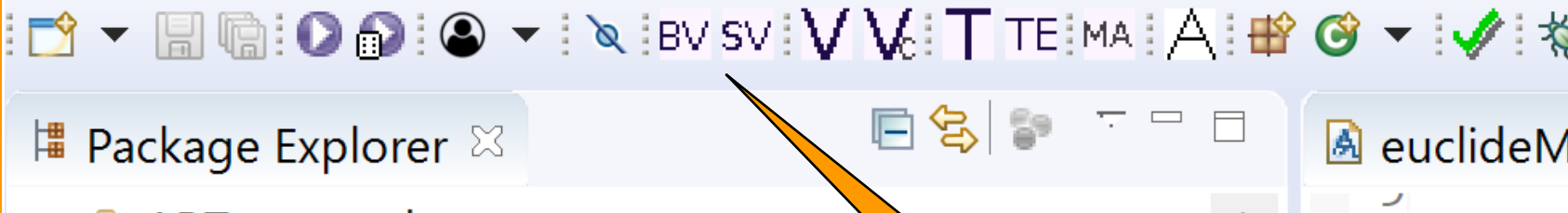
Rule	Visual tree	AsmetaL notation
Skip rule do nothing		skip
Update rule update f to v		$f := v$
Macro call rule		r_rule[]
invoke rule r_rule with arguments $\bar{v}$ (if any)		r_rule[$\bar{v}$]
Conditional rule execute $rule1$ if $guard$ holds, otherwise execute $rule2$ (if given)		if guard then rule1 endif
		if guard then rule1 else rule2 endif

Notazione grafica

Block rule execute <i>rule1</i> ... <i>ru_n</i> in parallel		par rule1 rule2 ... ru_n endpar
Forall rule execute <i>rule1</i> with all values $\bar{v} \in \bar{V}$ for which $d(\bar{v})$ holds		forall $\bar{v} \in \bar{V}$ with $d(\bar{v})$ do rule1[$\bar{v}$]
Choose rule execute <i>rule1</i> with a $\bar{v} \in \bar{V}$ for which $d(\bar{v})$ holds. If no such $\bar{v}$ ex- ists, execute <i>rule2</i> (if given)	 	choose $\bar{v} \in \bar{V}$ with $d(\bar{v})$ do rule1[$\bar{v}$] choose $\bar{v} \in \bar{V}$ with $d(\bar{v})$ do rule1[$\bar{v}$] ifnone rule2
Let rule execute <i>rule1</i> substi- tuting $\bar{t}$ for $\bar{x}$		let ($\bar{x} = \bar{t}$) in rule1[$\bar{x}$] endlet

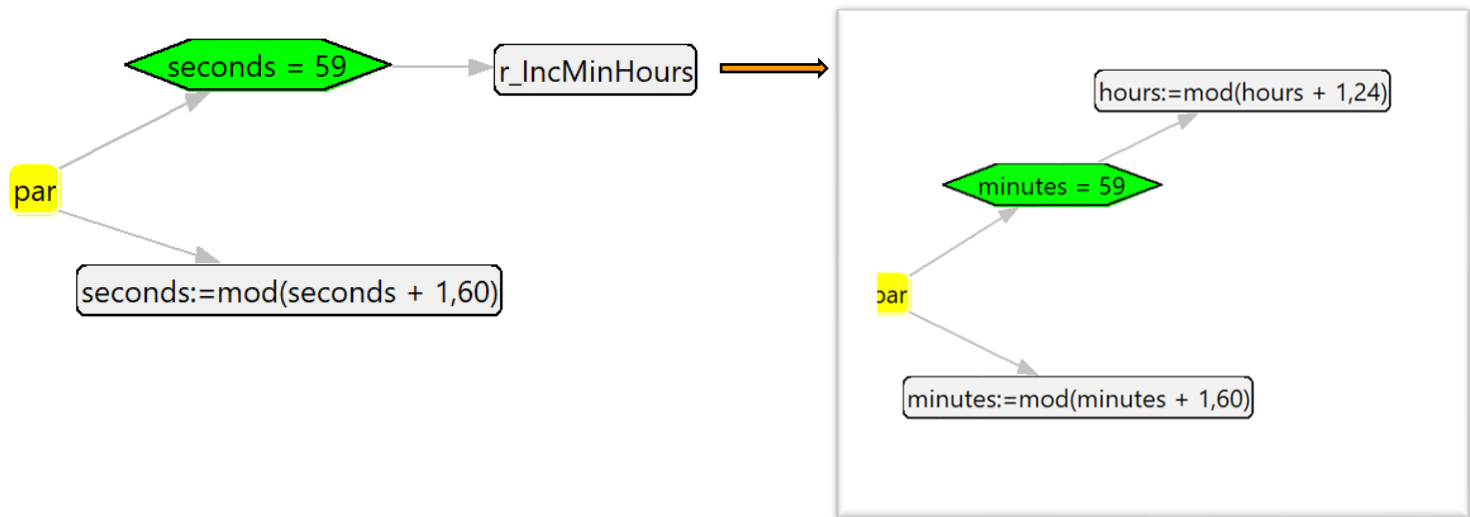
Avviare il visualizzatore

eclipse-workspace - asmetacode/euclideMCD.asm - Eclipse IDE
File Edit Navigate Search Project AsmEE Run Asmeta Window Help

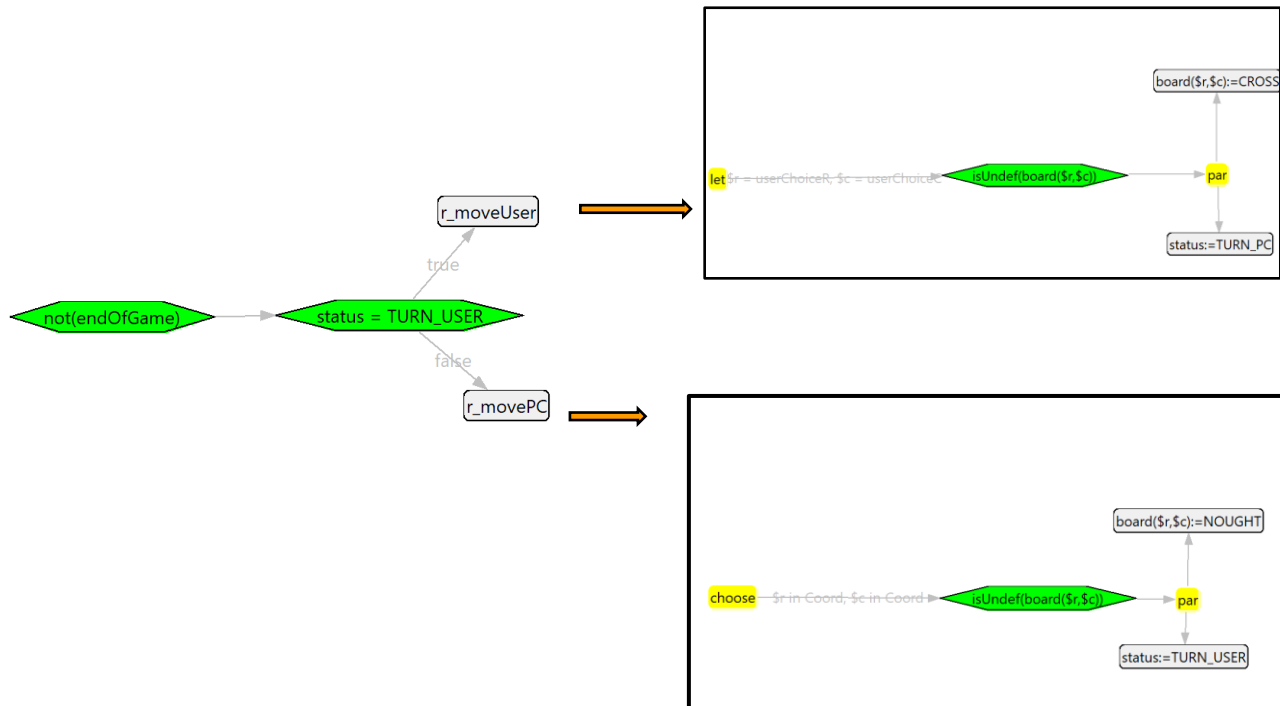


BV: basic
SV: semantic

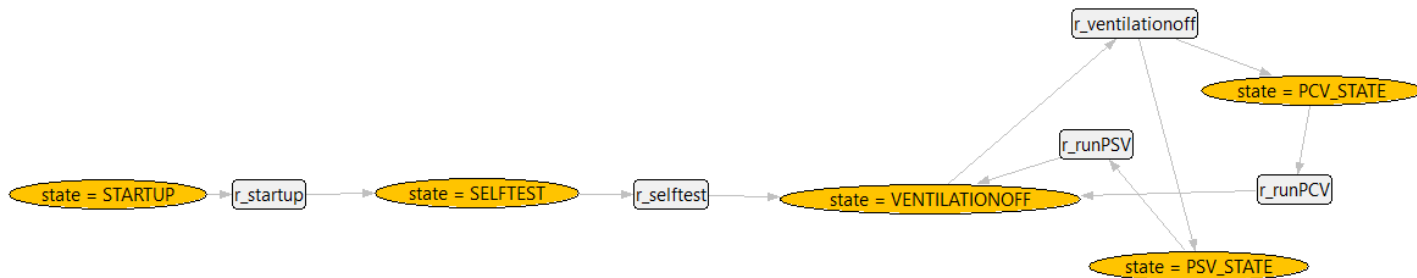
Advanced clock model



Tic Tac Toe visualization



Modello semantico: controller MVM



main rule r_Main =

par

//transition from startup to selftest

if state = STARTUP then

r_startup[]

endif

//transition from selftest to ventilation off

if state = SELFTEST then

r_selftest[]

endif

//start ventilation, either pcv or psv

if state = VENTILATIONOFF then

r_ventilationoff[]

endif

//transition from PCV to ventilation off if requested

if state = PCV_STATE then

r_runPCV[]

endif

//transition from PSV to ventilation off if requested

if state = PSV_STATE then

r_runPSV[]

endif

Esempi

- Euclide MCD
- Ferryman
- Fattoriale
- Tic Tac Toe
- Vending machine

Model Advisor

- Ha lo scopo di permettere *model review*, ossia un processo di revisione che dia garanzia della *qualità del modello*
- La qualità è misurata in termini di *attributi*,
 - caratteristiche che il modello deve avere come ASM, indipendentemente dai reqs
- Per ogni attivuto è definita una *meta-proprietà*
 - la sua violazione indica la presenza di vizi effettivi, o solo potenziali

Model Advisor

- Meta-proprietà sono definite in termini di *formule logiche temporali*
 - **MA richiede il model checker NuSMV**
- mp1 : No inconsistent update is ever performed
- mp2 : Every conditional rule must be complete
- mp3 : Every rule can eventually fire
- mp4 : No assignment is always trivial
- mp5 : For every domain element e there exists a location which has value e
- mp6 : Every controlled function can take any value in its co-domain
- mp7 : Every controlled location is updated and every location is read

Model Advisor

□ **Coerenza**

- garantisce assenza di aggiornamenti inconsistenti (MP1)

□ **Completezza**

- ogni comportamento del sistema sia modellato in modo esplicito: completezza di regole condizionali (MP2) ed effettivo aggiornamento delle lunzioni controllate (MP7)

□ **Minimalità** (no over-specification)

- No elementi definiti/dichiarati ma mai utilizzati (MP3, MP4, MP5, MP6); solo funzioni necessarie (MP7)